

Rapport sur le Développement de DataDetective

Introduction

Ce rapport détaille l'architecture du code, la répartition des tâches de développement et les méthodologies de test employées. Il expose les limitations et les bugs connus afin de garantir la maintenabilité et l'évolutivité du projet.

L'approche adoptée repose sur des principes rigoureux de développement, visant à produire un code lisible et modulaire. L'application stricte des bonnes pratiques facilite l'intégration de nouvelles fonctionnalités, tout en minimisant les erreurs et les incohérences structurelles.

1. Architecture du Code

1.1 Respect des Normes PEP8 et PEP257

L'architecture du code suit les standards Python, notamment PEP8 et PEP257, afin d'assurer une lisibilité et une structure cohérente.

- **PEP8 (Python Enhancement Proposal 8)** définit les règles de style, incluant l'indentation, les conventions de nommage et la gestion des importations, garantissant un code homogène et plus facile à relire.
- **PEP257 (Python Enhancement Proposal 257)** formalise l'utilisation de docstrings pour expliciter la finalité des modules, classes et fonctions, facilitant la compréhension du code.

1.2 Conception Modulaire

La modularité constitue un axe central de notre architecture. Chaque fonction possède une responsabilité bien définie, favorisant la maintenabilité et l'évolutivité.

- **Réutilisabilité** : La division en fonctions indépendantes permet la réutilisation de composants, réduisant ainsi la redondance et le temps de développement.
- **Maintenance** : Les modifications sont confinées à des fonctions précises, limitant leur impact sur l'ensemble du système.
- **Scalabilité** : Une architecture modulaire permet d'ajouter des fonctionnalités sans perturber la structure existante.

1.3 Typage des Fonctions

L'utilisation des annotations de type (typage statique) en Python permet la robustesse du code, notamment en permettant une meilleure détection des erreurs à l'écriture et une compréhension plus claire du code.

- **Détection précoce des erreurs** : Le typage statique prévient les incohérences de données avant l'exécution.
- **Lisibilité accrue** : Les annotations rendent explicites les types de données manipulées. Elles rendent le code plus intuitif pour les développeurs.
- **Facilitation de la maintenance** : L'analyse statique des types optimise la refactorisation et préserve la cohérence du code sur le long terme.

1.4 Utilisation du module subprocess

L'intégration du module subprocess assure une interaction efficace avec les processus système.

- **Simplicité et contrôle** : Permet d'exécuter et de superviser des processus externes sans complexifier le code.
- **Fiabilité accrue** : Offre des mécanismes de gestion des erreurs et des flux de données standardisés.

2. Amélioration de la Modularité

Bien que le code soit déjà structurant, plusieurs axes d'amélioration sont envisagés :

- **Décomposition supplémentaire** : Certaines fonctions peuvent être fragmentées en sous-fonctions plus fines pour améliorer leur compréhension et leur testabilité.
- **Utilisation de design patterns** : L'adoption de patterns comme Factory, Strategy ou Observer renforcera la modularité et l'extensibilité.
- **Exploration de frameworks** : L'intégration de bibliothèques facilitant la modularité sera étudiée.

3. Expressions Régulières

3.1 Utilisation dans le projet

Les expressions régulières permettent une analyse précise et efficace des chaînes de caractères, notamment dans :

- **L'analyse des logs** : Extraction d'informations structurées depuis des fichiers texte volumineux.
- **La validation des données** : Vérification de formats (emails, numéros de téléphone, adresses IP).
- **L'extraction de données de partition** : Identification des offsets critiques via la fonction `find_basic_data_partition_offset`.

3.2 Perspectives d'Amélioration

- **Précision accrue** : Optimisation des motifs regex pour réduire les faux positifs et capturer les informations les plus pertinentes.
- **Élargissement des cas d'utilisation** : Extension de l'usage des regex à d'autres traitements textuels (fichiers de configuration, logs avancés).
- **Intégration avec d'autres technologies** : Combinaison avec XML et JSON pour un traitement de données plus puissant.

Répartition des Contributions

Chaque membre de l'équipe a travaillé sur des parties spécifiques, garantissant expertise et responsabilité :

- **Franck FERMAN** : Développement du script et rédaction du rapport
- **Dimitri CARLIER** : Développement de l'extraction des fichiers
- **Mathis LEJOSNE** : Développement de l'extraction des fichiers
- **Myriam BELAID** : Rédaction du rapport
- **Théo THEBAULT** : Rédaction du rapport
- **Brahim EL FIKHI** : Développement de l'extraction des fichiers

Tests Implémentés

4.1 Tests Actuels

Une batterie de tests et d'intégration valide les fonctions en couvrant des cas courants et des scénarios d'erreur anticipés :

- *Arguments incompatibles.*
- *Image invalide.*
- *Partition invalide.*
- *Partition sans données.*
- *Nom de dossier invalide.*

1. Arguments incompatibles :

```
[*]-[eve@drop]-[~/DataDetective/src/DataDetective]
$python3 DataDetective.py -i /home/eve/Downloads/disk3.001 --check-image --show-partitions
Error: --check-image and --show-partitions cannot be used together.
```

2. Image invalide :

```
[eve@drop]-[~/DataDetective/src/DataDetective]
$python3 DataDetective.py -i not_image.dd --check-image
Error validating image: Sector offset supplied is larger than disk image (maximum: 0)
```

3. Partition invalide :

```
[eve@drop]--[~/DataDetective/src/DataDetective]
$python3 DataDetective.py -i not_image.dd --show-partitions

Error validating image:
```

4. Partition sans données :

```
[eve@drop]--[~/DataDetective/src/DataDetective]
$python3 DataDetective.py -i /home/eve/Downloads/disk3.001 --show-files

Failed to find 'Basic data partition'.
```

5. Nom de dossier invalide :

```
[eve@drop]--[~/DataDetective/src/DataDetective]
$python3 DataDetective.py -i /home/eve/Downloads/ewf/disk.E01 --show-dir test

Directory not found.
```

Tests Futurs : L'implémentation de tests de performance et de sécurité est prévue. Cela inclura des tests de charge pour évaluer la robustesse du logiciel sous des conditions de stress et des tests de pénétration pour identifier les vulnérabilités potentielles.

4.2 Tests Actuels

Des tests supplémentaires sont prévus pour renforcer la stabilité du projet :

- **Tests de performance** : Évaluer la robustesse sous charge.
- **Tests de sécurité** : Identifier les vulnérabilités potentielles.

Cas Non Supportés

Le code actuel ne prend en charge que les systèmes d'exploitation linux. Ce périmètre sera élargi dans les versions futures.

Bugs Connus

Plusieurs bugs mineurs ont été identifiés, notamment dans la gestion des erreurs de syntaxe des expressions régulières. Des solutions sont à l'étude et seront implémentées dans les prochaines mises à jour.

Conclusion

La conception de DataDetective repose sur des principes rigoureux garantissant sa maintenabilité et son évolutivité. Le respect des standards Python, l'adoption d'une architecture modulaire et l'utilisation d'éléments comme les expressions régulières et subprocess permettent de produire un programme fiable et adaptable.

L'amélioration continue de la modularité et de la gestion des données textuelles constitue un axe de développement stratégique, garantissant une meilleure réponse aux exigences futures du projet.